🛡️ Релиз: Zetern LEP

📦 Версия: 1.0

🐧 Категория: Linux Executable Protector

🎯 Назначение: Комплексная защита Linux-приложений от реверс-инжиниринга, несанкционированной модификации и анализа внутренней логики.

🚀 О Zetern LEP

Zetern LEP — протектор исполняемых ELF-файлов для Linux, предназначенный для повышения устойчивости программного обеспечения к анализу, взлому и модификации. Он сочетает виртуализацию кода, многоуровневую обфускацию, защиту исполняемых данных и механизмы обнаружения вмешательства в работающий процесс.

⚡ Основные возможности

🧠 1. Виртуализация кода

Виртуализация — один из наиболее сложных уровней защиты, реализуемых в Zetern LEP. Выбранные критически важные функции преобразуются из нативных инструкций процессора в собственный проприетарный байт-код.

- 🔹 Встроенная виртуальная машина: в ELF-файл интегрируется специализированная виртуальная машина с собственным интерпретатором байт-кода.
- 🔹 Выполнение через интерпретатор: защищённые участки программы обрабатываются виртуальной машиной вместо непосредственного выполнения исходных нативных инструкций.
- 🔹 Усложнение реверс-инжиниринга: анализаторы вроде IDA Pro, Ghidra и Radare2 сталкиваются с дополнительным уровнем абстракции. Для восстановления логики защищённых функций необходимо исследовать устройство виртуальной машины, формат байт-кода и семантику его инструкций.

> 💡 Результат: восстановление исходной логики программы может потребовать значительно больше времени и усилий по сравнению с анализом обычного нативного ELF-файла.

🔐 2. Продвинутая нативная обфускация

Zetern LEP использует методы обфускации, направленные на усложнение статического и динамического анализа исполняемого кода.

- 🔀 Control Flow Flattening — преобразование потока управления. Стандартная структура ветвлений, циклов и условных переходов преобразуется в более сложную схему управления выполнением, затрудняя восстановление исходной логики функций.
- ⚙️ Мутация инструкций и замена операторов. Выбранные операции заменяются эквивалентными последовательностями инструкций, усложняя распознавание исходных конструкций программы автоматизированными средствами анализа.
- 🧩 Dead Code Insertion — внедрение фиктивного кода. Добавляются вспомогательные вычисления и отвлекающие конструкции, не изменяющие предусмотренный результат выполнения, но увеличивающие сложность анализа бинарного файла.

> 💡 Результат: усложнение восстановления алгоритмов, распознавания ключевых функций и автоматической декомпиляции защищённых участков.

🗃️ 3. Шифрование и сокрытие структуры ELF

Zetern LEP предусматривает механизмы защиты содержимого и внутренней структуры исполняемого файла, затрудняющие извлечение полезной информации при статическом анализе.

- 🔒 Шифрование сегментов и данных. Поддерживаемые участки кода и данных могут храниться в зашифрованном виде и расшифровываться в памяти по мере необходимости для выполнения.
- 🕵️ Защита информации об импортах. Сведения о внешних зависимостях и вызываемых функциях преобразуются или скрываются там, где это допускает формат ELF и выбранный механизм загрузки. Это усложняет анализ взаимодействия приложения с libc, libstdc++ и другими библиотеками.
- 🔑 Шифрование строк. Строковые литералы могут храниться в зашифрованном виде и восстанавливаться во время выполнения. Это затрудняет извлечение текстов, путей, внутренних сообщений и других строковых данных с помощью инструментов вроде strings.

- 🧹 Удаление таблиц символов (Strip Symbols). Из ELF-файла удаляются ненужные таблицы символов и доступные отладочные данные, включая имена функций, если они не требуются для работы приложения.

> 💡 Результат: сокращение объёма информации, доступной исследователю без предварительного анализа защитных механизмов.

🛡️ 4. Защита во время выполнения: Anti-Debugging и RASP

Zetern LEP включает механизмы обнаружения отладки и вмешательства в работающий процесс, направленные на повышение устойчивости защищённого приложения к динамическому анализу.

- 🚫 Anti-Debugging — обнаружение отладчиков. Проверяется наличие признаков отладки, включая использование системных механизмов Linux, таких как ptrace. При обнаружении подозрительной активности приложение может завершить работу или задействовать предусмотренную реакцию защиты.
- 🔍 Memory Integrity Check — контроль целостности памяти. Содержимое выбранных участков исполняемой памяти периодически проверяется с помощью контрольных хешей. Обнаружение несоответствий позволяет выявлять некоторые виды изменения кода во время выполнения, включая попытки патчинга.
- 💾 Anti-Memory Dump — противодействие снятию дампов памяти. Применяются механизмы, повышающие сложность извлечения полного расшифрованного образа приложения из оперативной памяти. Это затрудняет анализ кода после его восстановления в процессе выполнения.

> 💡 Результат: повышение сложности динамического анализа, вмешательства в выполнение и обхода реализованных механизмов защиты.

🏆 Почему Zetern LEP?

✅ Виртуализация выбранных участков кода

✅ Многоуровневая нативная обфускация

✅ Шифрование строк и поддерживаемых участков данных

✅ Сокрытие информации, упрощающей статический анализ

✅ Обнаружение отладки и проверка целостности памяти

✅ Дополнительные барьеры против патчинга и извлечения кода

🚀 Zetern LEP 1.0 — защити свой код!

Zetern LEP объединяет виртуализацию, обфускацию, шифрование и механизмы контроля выполнения в едином решении для защиты Linux-приложений.

> 🎯 Главная цель: повысить стоимость реверс-инжиниринга, усложнить несанкционированное изменение программы и обеспечить дополнительные уровни защиты критически важных участков кода.

> ⚠️ Эффективность защиты зависит от конкретного приложения, выбранных параметров протекции и реализации отдельных механизмов. Ни один метод не гарантирует абсолютную защиту от взлома или реверс-инжиниринга.

> ✅Рекомендуемая команда сборки для максимальной защиты:
> `./zep app -o app-protected -l 3 --vm-all --vault-all --anti-debug 2 --kill 2`

> 🔑License key можно купить в личных сообщениях канала. Цена: 50$ (Навсегда). Оплата может проходить через CryptoBot (USDT) или же по платёжной ссылке Paddle.

> 🔰Сообщить о багах также можно в личных сообщениях канала - t.me/zeternn